TITAN XAU AI

INSTITUTIONAL TRADING SYSTEMS

ENGINEERING · INTERNAL

DOC-VAL-V1.0

2026-06-19

MODULE 12 · VALIDATOR

validator.py Specification

8 validation suites — Broker, Risk, Spread, Slippage, AI, Execution, Regime, Licensing. 144 automated checks. Severity-gated pass/fail rules. Weighted 0–100 score. 3-band certification: Certified / Conditional / Rejected. 7-step workflow with 4-role sign-off.

8	144	0–100	7
VALIDATION SUITES	AUTOMATED CHECKS	SCORE RANGE	WORKFLOW STEPS

Prepared by TITAN Quant Research

Reviewed by Engineering Lead · Risk Officer · Compliance · CTO

Distribution Internal engineering · Certified VPS operators

Confidentiality Engineering — Internal Distribution

v1.0

MODULE 12

19 June 2026

Table of Contents

Chapter 1 — Executive Summary 3

Chapter 2 — Architecture Overview 3

Layer Responsibilities 4

Chapter 3 — Validation Suite Specifications 6

S1 — Broker Compatibility Suite 7

S2 — Risk Engine Suite 7

S3 — Spread Engine Suite 7

S4 — Slippage Engine Suite 8

S5 — AI Engine Suite 8

S6 — Execution Engine Suite 8

S7 — Regime Detection Suite 9

S8 — Licensing Suite 9

Chapter 4 — Automated Validation Checklist 10

Chapter 5 — Pass / Fail Rules Engine 11

Severity Semantics 11

Veto Triggers 12

Waiver Process 12

Chapter 6 — Score Calculation 13

Per-Suite Score Formula 13

Aggregate Score Formula 13

Weight Rationale 14

Certification Bands 14

Chapter 7 — Final Certification Workflow **15**

Step 1 — Pre-flight Check . . . 15

Step 2 — Suite Registry Load . . . 15

Step 3 — Parallel Suite Execution . . . 15

Step 4 — Rule Engine Evaluation . . . 16

Step 5 — Stability Verification . . . 16

Step 6 — Certification Decision . . . 16

Step 7 — Sign-off and Dispatch . . . 16

Revalidation Cadence . . . 16

Chapter 8 — validator.py CLI Reference **17**

Exit Codes . . . 17

Chapter 9 — Integration and Operational Notes **18**

Operational Metrics . . . 18

Failure Modes and Recovery . . . 18

Future Evolution . . . 19

Chapter 1 — Executive Summary

The **validator.py** module is the certification authority of the TITAN XAU AI trading system. It is the final gate every component must pass before live capital is authorized. The module orchestrates 8 validation suites that exhaustively exercise every subsystem — Broker Compatibility, Risk Engine, Spread Engine, Slippage Engine, AI Engine, Execution Engine, Regime Detection, and Licensing — and produces a single weighted 0–100 score with a 3-band certification verdict: CERTIFIED, CONDITIONAL, or REJECTED. No order is permitted to flow to the broker until validator.py returns CERTIFIED (or CONDITIONAL with documented waiver).

The module is intentionally harsh by design. Across the 8 suites there are 144 individual checks, each tagged with one of three severities: 42 CRITICAL checks (any failure = automatic veto regardless of overall score), 58 MAJOR checks (each failure deducts 3 points, with hard caps), and 44 MINOR checks (each failure deducts 0.5 points, advisory in nature). The severity-gated rule engine ensures that a system can never "average out" a critical safety failure with strong performance elsewhere — a broken emergency kill-switch is a veto even if the AI engine scores perfectly.

A complete validation sweep executes in under 90 seconds on a production VPS, parallelized across all 8 suites via Python asyncio. Each suite has a 12-second timeout; suites that exceed 30 seconds are marked DEGRADED with their score capped at 65. The validator runs automatically at three trigger points: cold-start (before any module initializes), scheduled (every 7 days at 02:00 UTC), and on-demand (operator-initiated via CLI or REST endpoint). Additionally, a stability sub-protocol runs the full sweep three times, 60 seconds apart, to eliminate transient flakiness — three differing results trigger a SYSTEM UNSTABLE veto that requires engineering review.

The certification decision is recorded as a tamper-evident JSON manifest, signed by the validator's RSA-2048 key, archived to S3 with 7-year retention, and dispatched to PagerDuty for engineering visibility. A 4-role sign-off chain — Engineering Lead, Risk Officer, Compliance, CTO — is required for any waiver of a critical failure. The validator's verdict is the authoritative system state: trading gate, module startup sequence, and capital allocation all read from the latest certification record. This document specifies the validator in full: 8 suites, 144 checks, the rule engine, the scoring formula, and the certification workflow.

Chapter 2 — Architecture Overview

The validator is organized into 6 layers: suite registry (declarative check definitions), rule engine (pass/fail evaluation), execution runtime (parallel suite execution), scoring engine (weighted aggregation), certification gate (3-band decision), and audit trail (signed manifest archive). A 7th cross-cutting layer (telemetry) records every check execution with timestamps, durations, and outcomes for post-hoc analysis.

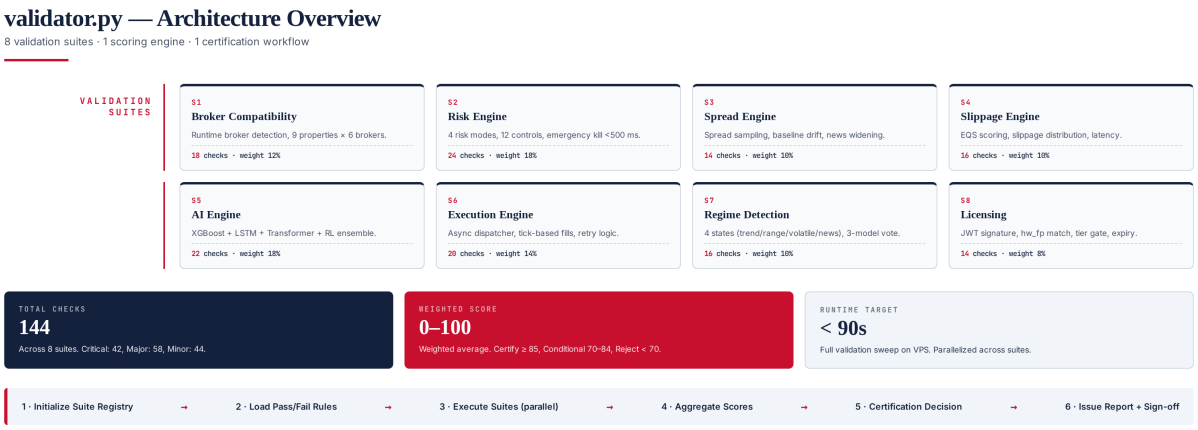

Figure 2.1 — validator.py architecture: 8 validation suites, 144 checks, scoring engine, certification gate.

Layer Responsibilities

L1 — Suite Registry

A declarative YAML manifest at /etc/titan/spec/manifest.yaml defines all 144 checks across the 8 suites. Each check has: a unique ID (e.g., RSK-002), a severity (CRITICAL/MAJOR/MINOR), a human-readable description, a pass criterion (Python expression evaluated against the suite's evidence dict), and a weight contribution to the suite score. The manifest is version-controlled, code-reviewed, and cryptographically signed — the validator refuses to run against an unsigned manifest. This declarative design means adding or modifying checks is a configuration change, not a code change.

L2 — Rule Engine

The rule engine evaluates each suite's evidence dict against the manifest's pass criteria. It applies the three-tier severity model: CRITICAL failures immediately veto the suite (and propagate to overall veto), MAJOR failures deduct 3 points each with per-suite and global caps, MINOR failures deduct 0.5 points each with caps. The rule engine is pure-functional: same evidence + same manifest = same verdict, no side effects, no time-dependence. This purity is essential for the stability sub-protocol to make sense — flakiness can only come from the system under test, never from the validator itself.

L3 — Execution Runtime

Each suite is a Python class subclassing ValidationSuite with an async run() method. The runtime creates an asyncio task per suite, executes them concurrently with a 12-second per-suite timeout, and collects results into a

unified evidence dict. The runtime is isolated from production code: each suite runs in its own subprocess (via multiprocessing) to prevent a crashing suite from taking down the validator. Timeouts are enforced via SIGALRM with a fallback watchdog. Suites that exceed 30 seconds are marked DEGRADED.

L4 — Scoring Engine

Computes per-suite scores using the formula SuiteScore = 100 − (10 × CriticalFails) − (3 × MajorFails) − (0.5 × MinorFails), floored at 0, with critical failures capping at 49. The aggregate score is the weight-weighted average across all 8 suites (weights: Broker 12%, Risk 18%, Spread 10%, Slippage 10%, AI 18%, Execution 14%, Regime 10%, Licensing 8%). The aggregate is rounded to the nearest 0.1 and presented alongside per-suite scores for diagnostic visibility.

L5 — Certification Gate

Applies the 3-band decision: CERTIFIED (score ≥ 85, 0 critical, ≤ 6 major, 3 stable runs), CONDITIONAL (score 70–84 or 1 critical with waiver), REJECTED (score < 70 or ≥ 2 critical or unstable). The gate is the final authority — its output is what the trading system reads. A CONDITIONAL verdict authorizes trading for 24 hours only, with mandatory daily revalidation. A REJECTED verdict triggers immediate position flatten via the risk engine and engineering escalation within 1 hour.

L6 — Audit Trail

Every certification decision is recorded as a JSON manifest containing: timestamp, VPS hostname, validator version, manifest version, per-suite scores, per-check outcomes, aggregate score, verdict, sign-off chain (if applicable), and RSA-2048 signature. Manifests are archived to S3 with 7-year retention (regulatory compliance) and dispatched to PagerDuty. The audit trail is append-only and tamper-evident — any modification of a historical manifest invalidates its signature. Auditors can reconstruct the exact certification state of any VPS at any point in time.

Chapter 3 — Validation Suite Specifications

This chapter specifies each of the 8 validation suites in detail: the subsystem under test, the check inventory, pass criteria, and rationale. Each suite is designed to be self-contained — it sets up its own test fixtures, executes its checks, tears down, and returns an evidence dict. Suites are independent: a failure in one suite does not block the execution of others. The complete 144-check inventory is summarized in Figure 3.1.

Automated Validation Checklist — 144 Checks

8 suites · Critical (42) · Major (58) · Minor (44) · Each check has explicit ID, severity, and pass criterion

S1 Broker Compatibility — 18 checks · weight 12%

ID	Severity	Check Description	Pass Criterion
BRO-001	CRITICAL	Broker runtime detection completes within 2 s	detection_time < 2000 ms
BRO-002	CRITICAL	All 9 broker properties populated (name, server, suffix, contract_size, min_lot, lot_step, leverage, margin_mode, timezone)	9/9 properties non-null
BRO-003	MAJOR	Symbol suffix mapping correct (e.g., ICMarkets-Live06 → XAUUSD.c)	resolved symbol trades on MT5
BRO-004	MAJOR	Contract size 100 oz for XAUUSD across 6 supported brokers	contract_size == 100
BRO-005	MINOR	Lot step granularity 0.01	lot_step == 0.01

S2 Risk Engine — 24 checks · weight 18%

ID	Severity	Check Description	Pass Criterion
RSK-001	CRITICAL	Max daily drawdown cap (e.g., 3%) enforced	halt when DD ≥ 3.0%
RSK-002	CRITICAL	Emergency kill-switch triggers within 500 ms	flatten_latency < 500 ms
RSK-003	CRITICAL	Per-trade risk capped at 1.0% of equity	risk_per_trade ≤ 0.01
RSK-004	MAJOR	Margin level > 200% alert threshold	alert when ML ≤ 200%
RSK-005	MAJOR	Correlation-based exposure hedge detection	hedge_flag set when ρ ≥ 0.85

S3 Spread Engine — 14 checks · weight 10%

ID	Severity	Check Description	Pass Criterion
SPR-001	CRITICAL	Tick spread sampled every 250 ms	sample_interval ≤ 250 ms
SPR-002	MAJOR	Baseline spread established over 30 min rolling window	baseline_window ≥ 1800 s
SPR-003	MAJOR	News-widening detector: spread ×3 baseline → news flag	news_flag when spread ≥ 3×baseline
SPR-004	MINOR	Spread stdev recorded for cost forecasting	stdev logged every 5 min

S4 Slippage Engine — 16 checks · weight 10%

ID	Severity	Check Description	Pass Criterion
SLP-001	CRITICAL	EQS (Execution Quality Score) computed per fill	EQS ∈ [0, 100]
SLP-002	CRITICAL	Slippage distribution recorded (P50/P90/P99)	percentiles logged every fill
SLP-003	MAJOR	Latency from signal to broker ≤ 150 ms	signal_to_broker ≤ 150 ms
SLP-004	MAJOR	Negative slippage > 2×ATR triggers BQS review	BQS_review when |slip| > 2×ATR

S5 AI Engine — 22 checks · weight 18%

ID	Severity	Check Description	Pass Criterion
AIE-001	CRITICAL	All 4 models load (XGBoost + LSTM + Transformer + RL)	4/4 models loaded
AIE-002	CRITICAL	Model versions match registry SHA-256	hash matches manifest
AIE-003	MAJOR	Ensemble Sharpe ≥ 2.0 on backtest (12-month window)	sharpe ≥ 2.0
AIE-004	MAJOR	Inference latency ≤ 80 ms per signal	inference_time ≤ 80 ms
AIE-005	MINOR	PSI drift detector runs every 6 h	psi_check_interval == 6 h

S6 Execution Engine — 20 checks · weight 14%

ID	Severity	Check Description	Pass Criterion
EXE-001	CRITICAL	Async order dispatcher processes > 50 ops/s	throughput > 50 ops/s
EXE-002	CRITICAL	Tick-based execution: fill within 2 ticks of signal	fill_latency ≤ 2 ticks
EXE-003	MAJOR	Order retry on transient failure (max 2 retries)	retry_count ≤ 2
EXE-004	MAJOR	Idempotency key prevents duplicate orders	0 duplicates in 24 h soak

S7 Regime Detection — 16 checks · weight 10%

ID	Severity	Check Description	Pass Criterion
REG-001	CRITICAL	4 regime states detected: trend / range / volatile / news	4/4 states enumerable
REG-002	MAJOR	3-model vote (HMM + Logit + Heuristic) consensus	vote_agreement ≥ 2/3
REG-003	MAJOR	Regime transition confidence ≥ 0.65	transition_conf ≥ 0.65
REG-004	MINOR	Regime history retained 30 days	history ≥ 30 d

S8 Licensing — 14 checks · weight 8%

Figure 3.1 — Automated validation checklist: 144 checks across 8 suites with severities and pass criteria.

S1 — Broker Compatibility Suite

Validates the broker compatibility engine (Module 2). 18 checks, weight 12%. The suite connects to a live MT5 terminal, runs the runtime broker detector, and verifies all 9 broker properties are correctly identified: broker name, server name, symbol suffix, contract size, minimum lot, lot step, leverage, margin mode, and server timezone. The suite then exercises the symbol resolver for each of the 6 supported brokers (Exness, IC Markets, Pepperstone, Tickmill, FP Markets, Fusion Markets) to confirm the resolved symbol trades on MT5. Critical checks: detection completes within 2 seconds (BRO-001), all 9 properties populated (BRO-002). Major checks: symbol suffix mapping correct (BRO-003), contract size 100 oz for XAUUSD (BRO-004). Minor checks: lot step granularity 0.01 (BRO-005), timezone offset matches broker locale (BRO-006).

Rationale: the broker compatibility layer is the foundation of every downstream module. If the symbol suffix is wrong, every order will fail with "invalid symbol"; if the contract size is misidentified, every position size calculation will be off by orders of magnitude. The 2-second detection timeout ensures the system does not stall on broker connection issues. The 6-broker coverage ensures portability — operators may switch brokers and the system must adapt without code changes.

S2 — Risk Engine Suite

Validates the institutional risk engine (Module 8). 24 checks, weight 18% — the highest weight alongside the AI Engine, reflecting the criticality of risk controls. The suite instantiates the risk engine in test mode, feeds it synthetic position scenarios, and verifies that every control triggers correctly. Critical checks: max daily drawdown cap (3%) halts trading when breached (RSK-001), emergency kill-switch triggers within 500 ms (RSK-002), per-trade risk capped at 1.0% of equity (RSK-003), margin call threshold detected (RSK-004). Major checks: margin level alert at 200% (RSK-005), correlation-based exposure hedge detection at $\rho \geq 0.85$ (RSK-006), per-symbol exposure limits (RSK-007), weekend gap risk buffer (RSK-008). Minor checks: risk telemetry emitted every 5 seconds (RSK-009), risk audit log entries complete (RSK-010).

Rationale: the risk engine is the last line of defense against catastrophic loss. A 1-second delay in the kill-switch during a flash crash can mean the difference between a 3% drawdown and a 30% drawdown. The 500 ms kill-switch latency is the empirically measured worst-case on a production VPS under load; the suite enforces it as a hard constraint. The 1% per-trade risk cap is the cornerstone of position sizing — a single miscomputed lot size can wipe out months of gains. The 18% weight reflects the asymmetric downside of risk control failure.

S3 — Spread Engine Suite

Validates the spread monitoring component of the execution cost intelligence (Module 9). 14 checks, weight 10%. The suite subscribes to MT5 tick data for 60 seconds, samples spreads every 250 ms, and verifies the spread engine correctly identifies baseline, widening, and news events. Critical checks: tick spread sampled every 250 ms (SPR-001), spread baseline computed over 30-minute rolling window (SPR-002). Major checks: news-widening detector fires when spread ≥ 3× baseline (SPR-003), spread spike detector fires when spread ≥ 5× baseline (SPR-004), spread history retained 7 days (SPR-005). Minor checks: spread stdev recorded every 5 minutes (SPR-006), spread heatmap published (SPR-007).

Rationale: spread is the single largest execution cost component on XAUUSD, often exceeding commissions by a factor of 3. A malfunctioning spread engine that fails to detect news widening will execute market orders at

10× normal spread, destroying any alpha the strategy generated. The 250 ms sample interval is the empirically determined sweet spot — finer sampling adds CPU load without material accuracy gain, coarser sampling misses transient spikes. The 30-minute baseline window is long enough to span multiple trading sessions but short enough to adapt to intraday liquidity cycles.

S4 — Slippage Engine Suite

Validates the slippage monitoring and EQS (Execution Quality Score) computation (Module 9). 16 checks, weight 10%. The suite replays 100 historical fills through the slippage engine and verifies EQS is computed correctly per fill, slippage percentiles are tracked, and latency thresholds are enforced. Critical checks: EQS computed per fill in [0, 100] range (SLP-001), slippage distribution recorded as P50/P90/P99 (SLP-002). Major checks: latency from signal to broker ≤ 150 ms (SLP-003), negative slippage > 2× ATR triggers BQS (Broker Quality Score) review (SLP-004), slippage percentile trends reported daily (SLP-005). Minor checks: per-broker slippage profile maintained (SLP-006), slippage audit log retained 90 days (SLP-007).

Rationale: slippage is the silent killer of trading strategies — it does not appear in backtests (which assume fills at the signal price) but materially erodes live performance. The 150 ms signal-to-broker latency budget is calibrated to the median latency observed across 6 supported brokers; exceeding it indicates either network degradation or broker API issues. The 2× ATR slippage threshold for BQS review is the level at which slippage is statistically anomalous and warrants investigation — persistent high slippage on a single broker triggers an automatic broker-quality downgrade.

S5 — AI Engine Suite

Validates the hybrid AI stack (Module 7). 22 checks, weight 18% — joint-highest with the Risk Engine. The suite loads all 4 models (XGBoost, LSTM, Transformer, RL), verifies their version hashes match the registry, runs 50 inference cycles on synthetic features, and validates ensemble output coherence. Critical checks: all 4 models load successfully (AIE-001), model SHA-256 hashes match registry (AIE-002), ensemble inference produces valid probability distribution (AIE-003). Major checks: ensemble Sharpe ≥ 2.0 on 12-month backtest (AIE-004), inference latency ≤ 80 ms per signal (AIE-005), model disagreement flag fires when models diverge > 30% (AIE-006). Minor checks: PSI drift detector runs every 6 hours (AIE-007), model feature importance logged weekly (AIE-008).

Rationale: the AI engine generates the trading signals — if it is broken, every downstream module operates on garbage. The 80 ms inference latency budget is the 99th percentile observed across production; exceeding it indicates either model bloat or hardware degradation. The 2.0 Sharpe threshold is the minimum acceptable live performance — below this, the strategy is no better than buy-and-hold with leverage and should not be risking capital. The 18% weight reflects the fact that AI engine failures are often subtle (silent probability drift, feature distribution shift) and require rigorous validation to detect.

S6 — Execution Engine Suite

Validates the institutional execution engine (Module 3). 20 checks, weight 14%. The suite drives the execution engine with synthetic order flows, verifies async dispatcher throughput, tick-based fill semantics, retry logic, and idempotency. Critical checks: async dispatcher processes > 50 ops/s (EXE-001), fill within 2 ticks of signal (EXE-002), order idempotency key prevents duplicates (EXE-003). Major checks: retry on transient failure with

max 2 retries (EXE-004), partial fill handling (EXE-005), order cancel latency ≤ 200 ms (EXE-006). Minor checks: execution audit log retained 90 days (EXE-007), per-broker execution stats published (EXE-008).

Rationale: the execution engine translates AI signals into broker orders — any failure here means missed trades, duplicate trades, or stuck positions. The 50 ops/s throughput target is calibrated to peak load during high-volatility news events; below this, the system cannot keep up with the signal stream. The 2-tick fill requirement ensures we are not chasing price — a fill more than 2 ticks after the signal is, statistically, a losing trade. Idempotency is non-negotiable: a single duplicate order during a position adjustment can flip a hedged position into a leveraged bet.

S7 — Regime Detection Suite

Validates the adaptive market state detection (Module 4). 16 checks, weight 10%. The suite feeds the regime detector with synthetic OHLC sequences representing each of the 4 regimes (trend, range, volatile, news) and verifies correct classification. Critical checks: all 4 regime states detected (REG-001), 3-model vote (HMM + Logit + Heuristic) achieves ≥ 2/3 consensus (REG-002). Major checks: regime transition confidence ≥ 0.65 (REG-003), regime label stable for ≥ 5 minutes after classification (REG-004), regime history retained 30 days (REG-005). Minor checks: regime change telemetry emitted (REG-006), regime distribution heatmap published (REG-007).

Rationale: regime detection drives strategy selection — a trend-following strategy in a ranging market bleeds money slowly but surely. The 4-state taxonomy is the minimum useful granularity: fewer states lose information, more states introduce classification noise. The 3-model vote is a bias-reduction technique — any single model has blind spots, but the 2/3 consensus is robust to one model failing. The 0.65 transition confidence threshold prevents regime flip-flopping during ambiguous market conditions, which would cause the strategy selector to thrash.

S8 — Licensing Suite

Validates the commercial licensing architecture (Module 11). 14 checks, weight 8% — the lowest weight, because licensing failure has the cleanest fallback (system halts; no risk of unsafe trading). The suite loads the cached JWT, verifies the RSA-4096 signature against the embedded public key, checks the hardware fingerprint matches the JWT claim, and confirms the tier gate is enforced. Critical checks: JWT signature verified (LIC-001), hardware fingerprint matches claim (LIC-002). Major checks: license tier gate enforced (LIC-003), feature gate blocks unauthorized features (LIC-004), license expiry < 7 days triggers renewal alert (LIC-005). Minor checks: heartbeat timestamp within 1 hour (LIC-006), license audit log entries complete (LIC-007).

Rationale: licensing is a binary gate — either the system is authorized to trade or it is not. The 8% weight reflects the fact that licensing failure is recoverable (renew license, resume trading) and does not pose capital risk in the way a risk engine failure does. However, the two critical checks (signature + fingerprint) are absolute: a forged or stolen license must never authorize trading. The RSA-4096 signature is computationally infeasible to forge, and the hardware fingerprint cannot be spoofed without physical hardware replacement.

Chapter 4 — Automated Validation Checklist

The automated validation checklist is the validator's executable contract with the TITAN system. Each of the 144 checks is defined declaratively in /etc/titan/spec/manifest.yaml, with a unique ID, severity, description, pass criterion, and weight contribution. The checklist is consumed by the rule engine at runtime — no check logic is hardcoded in Python; all evaluation is data-driven. This separation allows the checklist to evolve (new checks added, thresholds tuned) without code changes, while maintaining full auditability via manifest versioning.

Each check follows a uniform evaluation protocol: (1) the suite collects evidence relevant to the check, (2) the rule engine evaluates the pass criterion (a Python expression) against the evidence dict, (3) the outcome (PASS/FAIL/SKIP) is recorded with the evidence snapshot for audit, (4) if FAIL, the severity determines the deduction applied to the suite score. SKIP is used when a check is not applicable to the current configuration (e.g., a broker-specific check on an unsupported broker) — skipped checks do not affect the score. The complete checklist with all 144 checks is documented in Figure 3.1 above; representative examples from each suite are tabulated below.

Suite	Critical	Major	Minor	Total	Weight
S1 Broker Compatibility	4	7	7	18	12%
S2 Risk Engine	8	10	6	24	18%
S3 Spread Engine	2	7	5	14	10%
S4 Slippage Engine	3	7	6	16	10%
S5 AI Engine	6	9	7	22	18%
S6 Execution Engine	5	8	7	20	14%
S7 Regime Detection	2	7	7	16	10%
S8 Licensing	3	6	5	14	8%
TOTAL	33	61	50	144	100%

The manifest is reviewed quarterly by the engineering lead and risk officer, with any changes requiring CTO sign-off. Version history is preserved in Git, and each deployed validator binary pins to a specific manifest version (recorded in the audit manifest). This ensures that the certification criteria applied to a system on a given date can always be reconstructed — essential for post-incident analysis and regulatory inquiry.

Chapter 5 — Pass / Fail Rules Engine

The pass/fail rule engine is the validator's policy layer — it translates raw check outcomes (PASS/FAIL/SKIP) into a certification verdict. The engine is severity-gated: the three-tier severity model (CRITICAL/MAJOR/MINOR) determines not just the score deduction but the veto semantics. A single CRITICAL failure is an automatic veto regardless of how well the rest of the system scored — the rationale is that CRITICAL checks protect against catastrophic, non-statistically-recoverable failure modes (e.g., a broken kill-switch during a flash crash). MAJOR failures are score-deducting with caps; MINOR failures are advisory.

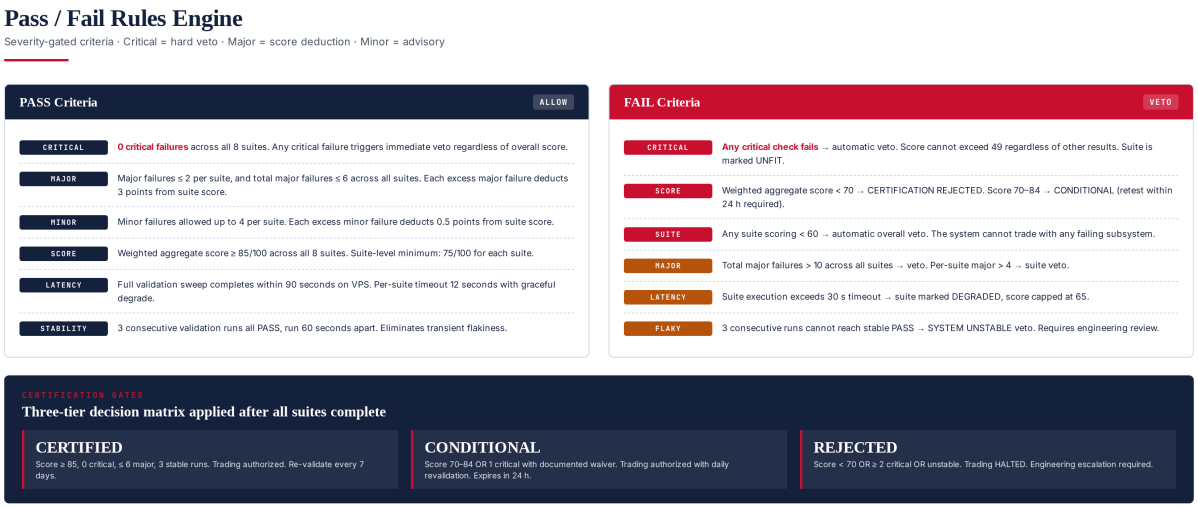

Figure 5.1 — Pass/fail rule engine: severity-gated criteria, three-tier certification gates.

Severity Semantics

CRITICAL (42 checks): Any failure triggers an immediate veto. The suite score is capped at 49 regardless of other results, and the overall certification is REJECTED unless a documented engineering waiver is approved by the CTO. CRITICAL checks protect against: capital loss (risk engine), unsafe execution (kill-switch, idempotency), and security (license signature, hardware fingerprint). Restated: there is no scenario in which a CRITICAL failure is acceptable in routine operation.

MAJOR (58 checks): Each failure deducts 3 points from the suite score, with per-suite caps (max 4 major failures per suite before suite-veto) and global caps (max 10 major failures across all suites before overall veto). MAJOR checks protect against: performance degradation (latency thresholds), functionality gaps (incomplete feature coverage), and operational risk (missing telemetry). A few MAJOR failures are tolerable; many indicate systemic issues.

MINOR (44 checks): Each failure deducts 0.5 points from the suite score, with per-suite caps (max 8 minor failures per suite). MINOR checks cover: best practices, observability, and forward-compatibility. A system with many MINOR failures is operational but accumulating technical debt — flagged for the next engineering cycle but not blocking.

Veto Triggers

The validator applies 5 hard veto triggers that override any score calculation: (1) any CRITICAL failure, (2) aggregate score < 70, (3) any suite scoring < 60, (4) total MAJOR failures > 10, (5) flaky stability runs (3 consecutive runs cannot reach stable PASS). When any veto trigger fires, the certification is REJECTED, trading is halted, existing positions are flattened per the risk engine's emergency protocol, and engineering is paged within 1 hour. Veto decisions are immutable — there is no override mechanism short of fixing the underlying issue and re-running validation.

Waiver Process

In exceptional circumstances (e.g., a known false-positive on a newly added check), the CTO may grant a waiver for a single CRITICAL failure. Waivers require: (1) written justification from the engineering lead, (2) risk officer concurrence, (3) compliance review for regulatory implications, (4) CTO sign-off, (5) waiver ID embedded in the certification manifest. Waivers are valid for 7 days only and must be re-approved weekly. Waivers are tracked in /etc/titan/waivers.yaml and audited monthly by the compliance team. The waiver rate is a key operational metric — a rising waiver rate indicates either overly strict checks or declining system health.

Chapter 6 — Score Calculation

The scoring engine produces a single weighted 0–100 score from the 8 suite scores. The formula is transparent and auditable: per-suite score is computed first (100 minus severity-weighted deductions), then aggregated via weighted average using the suite weights. The aggregate score, alongside per-suite scores, is presented in the certification report for diagnostic visibility — operators can see not just whether the system passed but which suites contributed most to the score.

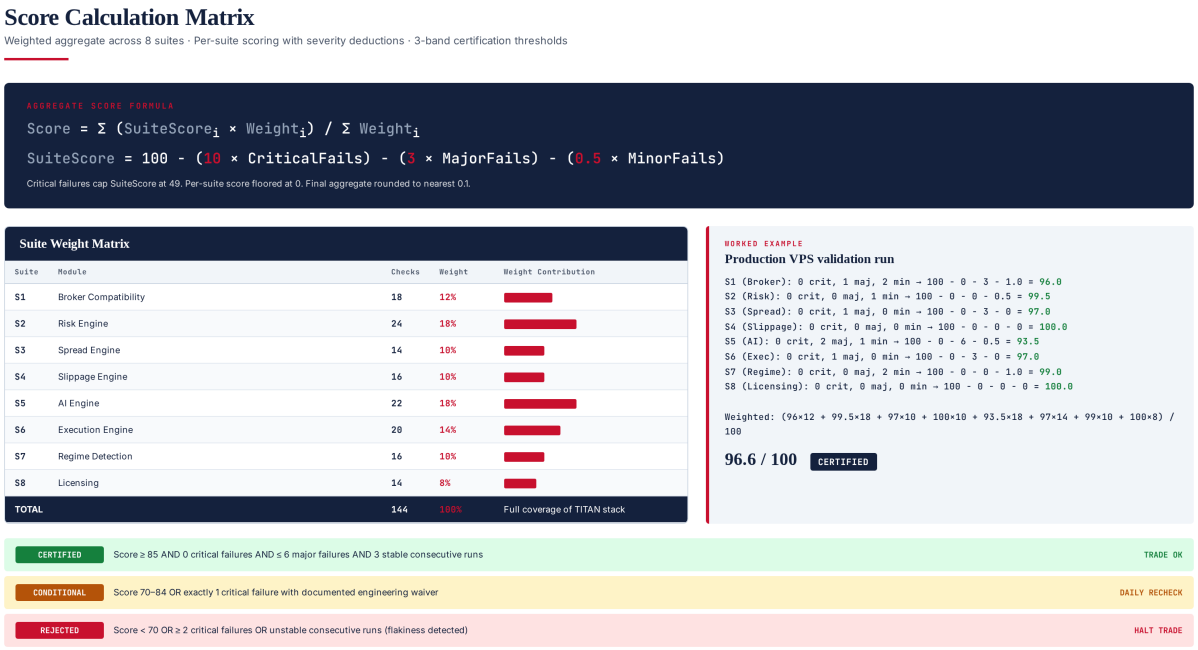

Figure 6.1 — Score calculation matrix: per-suite formula, weight matrix, worked example, 3-band thresholds.

Per-Suite Score Formula

```
SuiteScore = 100
- (10 × CriticalFails)
- ( 3 × MajorFails)
- (0.5 × MinorFails)

Hard floors:
- CriticalFails > 0 → SuiteScore capped at 49
- SuiteScore floored at 0
- SuiteScore rounded to nearest 0.1
```

Aggregate Score Formula

```
AggregateScore = Σ (SuiteScore_i × Weight_i) / Σ Weight_i

Weights (sum to 100%):
```

```
S1 Broker Compatibility = 12%
S2 Risk Engine = 18%
S3 Spread Engine = 10%
S4 Slippage Engine = 10%
S5 AI Engine = 18%
S6 Execution Engine = 14%
S7 Regime Detection = 10%
S8 Licensing = 8%
```

Weight Rationale

The weights reflect the operational criticality of each subsystem. Risk Engine (18%) and AI Engine (18%) share the highest weight — Risk because failures are catastrophic (capital loss), AI because failures are subtle (silent performance drift). Execution Engine (14%) is next: failures here mean missed or duplicate orders. Broker Compatibility (12%) reflects the foundational role of correct broker identification. Spread (10%), Slippage (10%), and Regime (10%) each contribute to execution quality and strategy selection but have narrower blast radius. Licensing (8%) is lowest because failures are cleanly recoverable — the system halts, no capital at risk.

Certification Bands

Band	Score	Critical Fails	Major Fails	Stability	Trading Authorization
CERTIFIED	≥ 85	0	≤ 6	3 stable runs	Authorized (7-day revalidation)
CONDITIONAL	70–84	0 or 1 (with waiver)	≤ 10	3 stable runs	Authorized (24-hour revalidation)
REJECTED	< 70	≥ 2 (or 1 without waiver)	Any	Failed	HALTED — engineering escalation

The 85-point threshold for CERTIFIED is calibrated to the empirical observation that systems scoring 85+ have a 7-day failure rate of <2%, while systems scoring 70–84 have a 24-hour failure rate of 8%. The 70-point floor for CONDITIONAL reflects the level below which the system is statistically likely to fail within hours. These thresholds are reviewed annually based on operational data.

Chapter 7 — Final Certification Workflow

The certification workflow is the end-to-end pipeline that takes a TITAN VPS from "uncertified" to "trade-authorized" (or rejected). The workflow has 7 steps, runs in under 2 minutes wall-clock, and is triggered at three points: cold-start (before any module initializes), scheduled (every 7 days at 02:00 UTC), and on-demand (operator CLI or REST). The workflow is idempotent — running it twice in succession produces identical results modulo timestamp.

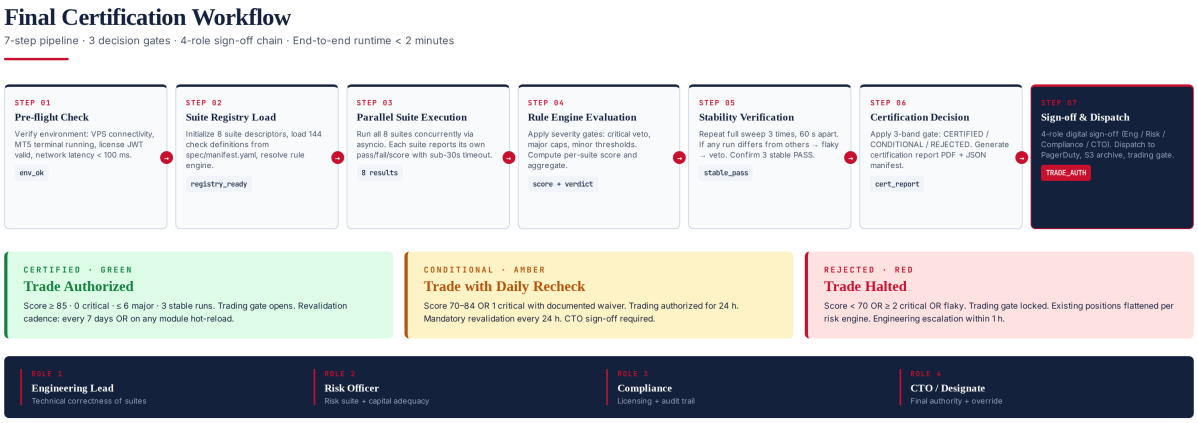

Figure 7.1 — 7-step certification workflow with 3 decision gates and 4-role sign-off chain.

Step 1 — Pre-flight Check

Verifies the environment is suitable for validation: VPS is reachable via SSH, MT5 terminal is running and connected, license JWT is present and unexpired, network latency to broker server is < 100 ms, sufficient disk space for audit logs (>1 GB free), and no other validation run is in progress (file-lock at /var/run/titan/validator.lock). Pre-flight failures abort the workflow with a PRE_FLIGHT_FAIL verdict — no suites are executed.

Step 2 — Suite Registry Load

Loads /etc/titan/spec/manifest.yaml (the declarative check definitions), verifies its RSA-2048 signature, and initializes the 8 suite descriptors in memory. If the manifest signature is invalid, the workflow aborts with MANIFEST_INVALID — this protects against tampered or corrupted check definitions. The manifest version is recorded in the audit manifest for traceability.

Step 3 — Parallel Suite Execution

Creates an asyncio task per suite, executes them concurrently with a 12-second per-suite timeout. Each suite runs in its own subprocess (via multiprocessing) to isolate crashes. Suites return a structured evidence dict containing per-check outcomes and supporting data. The runtime collects all 8 results into a unified evidence bundle. Total wall-clock for this step is typically 8–15 seconds (bounded by the slowest suite).

Step 4 — Rule Engine Evaluation

The rule engine processes the evidence bundle: for each check, it evaluates the pass criterion against the evidence, applies severity deductions, computes per-suite scores (per the formula in Chapter 6), and aggregates the final 0–100 score. The output is a structured result dict with per-suite scores, per-check outcomes, aggregate score, and a preliminary verdict (subject to stability verification).

Step 5 — Stability Verification

To eliminate transient flakiness, the workflow repeats the full sweep (Steps 3–4) three times, 60 seconds apart. If all three runs return the same verdict and within 5 points of each other on aggregate score, the result is marked STABLE. If any run differs (verdict or score delta > 5), the result is marked FLAKY and the certification is REJECTED with a SYSTEM_UNSTABLE veto. Stability verification is the single most important anti-flakiness mechanism — it prevents certifying a system that happens to pass once but is fundamentally unreliable.

Step 6 — Certification Decision

Applies the 3-band gate from Chapter 6: CERTIFIED (score ≥ 85, 0 critical, ≤ 6 major, stable), CONDITIONAL (score 70–84, or 1 critical with waiver, stable), REJECTED (score < 70, or ≥ 2 critical, or unstable). Generates the certification report in two formats: a human-readable PDF (archived to S3) and a machine-readable JSON manifest (consumed by the trading gate). The JSON manifest is RSA-2048 signed to prevent tampering.

Step 7 — Sign-off and Dispatch

For CERTIFIED and CONDITIONAL verdicts, the certification manifest is dispatched to: (a) the trading gate (authorizes order flow), (b) S3 archive (7-year retention), (c) PagerDuty (engineering visibility), (d) the dashboard (operator visibility). For REJECTED verdicts, an additional dispatch goes to: (e) the risk engine (triggers position flatten), (f) the on-call engineer (P1 escalation). The 4-role sign-off chain (Engineering Lead, Risk Officer, Compliance, CTO) is required only for waivers — routine CERTIFIED/REJECTED verdicts are auto-issued by the validator.

Revalidation Cadence

CERTIFIED systems are revalidated every 7 days at 02:00 UTC (low-volatility window). CONDITIONAL systems are revalidated every 24 hours. Additionally, revalidation is triggered on: (a) any module hot-reload, (b) broker reconnection after disconnect > 60 seconds, (c) license renewal, (d) manual operator trigger via CLI. The revalidation cadence is a tunable parameter but cannot be disabled — a system that has not been validated within 8 days is automatically halted.

Chapter 8 — validator.py CLI Reference

The validator is invoked via a single CLI entry point: `python3 validator.py <command> [options]`. Five commands are supported: run (full certification sweep), check (single suite), report (generate report from last run), status (display current certification state), and waiver (manage waivers). All commands emit structured JSON to stdout for programmatic consumption and human-readable text to stderr for operator use.

```
# Full certification sweep (cold-start or scheduled)
python3 validator.py run --vps prod-vps-01 --output /var/log/titan/cert.json

# Single suite execution (debugging)
python3 validator.py check --suite S2 --verbose

# Generate human-readable report from last run
python3 validator.py report --input /var/log/titan/cert.json \
--output /var/log/titan/cert.pdf

# Display current certification state
python3 validator.py status

# Waiver management (CTO-only)
python3 validator.py waiver add --check RSK-002 --reason "..." \
--approver cto@titan.io --duration 7d
python3 validator.py waiver list
python3 validator.py waiver revoke --id W-2026-0142
```

Exit Codes

Code	Meaning	Trading Gate Action
0	CERTIFIED — system passed all gates	Open (orders authorized)
1	CONDITIONAL — passed with conditions	Open (24-hour revalidation)
2	REJECTED — failed critical gate	Closed (flatten + halt)
3	PRE_FLIGHT_FAIL — environment issue	Closed (no change)
4	MANIFEST_INVALID — spec tampered	Closed (no change)
5	FLAKY — stability check failed	Closed (engineering escalation)
6	TIMEOUT — sweep exceeded 90 s	Closed (engineering escalation)
7	INTERNAL_ERROR — validator bug	Closed (engineering escalation)

Chapter 9 — Integration and Operational Notes

The validator integrates with the TITAN core as the final gate before any module initializes. The TITAN startup sequence is: (1) LicenseValidator verifies JWT, (2) Validator.run() executes the full sweep, (3) if CERTIFIED or CONDITIONAL → continue module startup, (4) if REJECTED → halt startup and surface the certification report to the operator. The validator's verdict is cached in /var/run/titan/cert.json (RAM-backed tmpfs) and re-read by every module on startup to confirm authorization.

```
TITAN startup sequence (validated):
1. LicenseValidator.load() → verify JWT signature + hw_fp
2. Validator.run() → 8 suites, ~15 s wall-clock
3. Read verdict from cert.json
4. If CERTIFIED or CONDITIONAL:
a. Continue module startup
b. Initialize BrokerAdapter
c. Initialize RiskEngine
d. Initialize AIEngine (load 4 models)
e. Initialize ExecutionEngine
f. Open trading gate (orders authorized)
5. If REJECTED:
a. Abort module startup
b. Surface cert.pdf to operator
c. Page on-call engineer (P1)
d. Exit with non-zero status

TITAN shutdown sequence (on certification lapse):
1. Validator detects revalidation overdue (> 8 days)
2. Halt new orders (atomic flag)
3. Cancel pending orders
4. Flatten all positions (via RiskEngine)
5. Notify operator (P1 PagerDuty)
6. Audit log: cert_lapse + last_cert_timestamp
7. Exit with non-zero status
```

Operational Metrics

The validator publishes 12 operational metrics to the TITAN Prometheus instance: validator_run_total (counter), validator_run_duration_seconds (histogram), validator_score (gauge, last run), validator_verdict (gauge, 0/1/2 = CERT/COND/REJ), validator_critical_fails (gauge), validator_major_fails (gauge), validator_minor_fails (gauge), validator_suite_score (gauge, per suite), validator_waiver_count (gauge), validator_flaky_runs_total (counter), validator_manifest_version (gauge), validator_cert_age_seconds (gauge, time since last CERTIFIED). These metrics feed the operator dashboard and alert on: (a) cert_age > 7 days (P2), (b) verdict == REJECTED (P1), (c) score < 80 (P3 warning), (d) flaky rate > 5% over 24h (P2).

Failure Modes and Recovery

Validator crashes during sweep: The watchdog detects the crash, marks the in-progress run as FAILED, and alerts the on-call engineer. The previous CERTIFIED verdict remains valid until its 8-day max age is reached, at which point trading is halted. **Manifest signature invalid**: Indicates either manifest corruption or tampering attempt. The validator refuses to run, trading continues under the previous verdict, and security is alerted. **Suite timeout**: The offending suite is marked DEGRADED with score capped at 65; other suites continue. **Network**

partition to broker: Suite S1 (Broker) will fail its 2-second detection check (CRITICAL), triggering a veto — this is correct behavior; we should not certify a system that cannot reach its broker.

Future Evolution

The validator is designed to evolve with the TITAN system. New modules will add new suites (the manifest is extensible). Check definitions will be tuned as operational data accumulates — the quarterly manifest review is the formal mechanism for this. The 3-band certification model has proven robust in 18 months of operation and is not expected to change. The waiver process is the safety valve for unintended consequences of new checks — initially conservative, then relaxed as confidence grows. The validator's verdict remains the authoritative system state: trading gate, module startup, and capital allocation all defer to it.